Regis University CPS
CS361 Programming Assignment 6
Due by midnight Sunday of week 6

NOTE: Remember that you are **NOT** allowed to use the **break, return,** or **exit** statements to exit from a loop. All loops must exit ***ONLY*** via their **test condition**.

Program #1

The local yogurt shop is expanding its selection of frozen treats, and would like you to modify the program you wrote in week 3 to calculate and print their customer's bills. You will also write a test plan to test the program.

- This shop now has four types of frozen treats: frozen yogurt and soft serve ice cream by the ounce, premium ice cream by the scoop, and ice cream bars. These items are available at the following rates:

Treat Type	Price
Frozen Yogurt	$ 0.44 per ounce
Soft Serve Ice Cream	$ 0.49 per ounce
Premium Ice Cream	$ 1.62 per scoop
Ice Cream Bars	$ 3.49 per bar

- Single servings of frozen yogurt or either type of ice cream may be purchased in a plastic cup (waffle cones are no longer available).
    - Plastic cups hold up to 16 ounces or 5 scoops
- Customers may optionally add toppings to their yogurt or ice cream. The first topping is included free, but additional toppings cost $0.63 per topping.
- Yogurt and soft serve ice cream may be purchased, without any toppings, in larger take out container that will hold amounts up to 64 ounces. Take out purchases are sold at a discount. (NOTE: If a purchase is over 16 oz, the program will be assume it is a take out purchase.)
    - Purchases (over 16 oz) up to 32 ounces are given a $0.04 discount per ounce.
    - Purchases of more than 32 ounces are given a $0.08 discount per ounce.
- Multiple treats may be purchased on a single bill.
- The punch card program has been discontinued in favor of a quantity discount. If 4 or more treats are purchased on one bill, the ***last*** treat in the purchase is charged half price.

Define constants for all fixed values, and use **double** precision floating point variables for all dollar & cents figures.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Implementation Details

At a **minimum**, the program must implement the following functions (in addition to **main**):

- Generic function 1 to read and validate any positive number (including 0).

  This function will have one input parameter, a string prompt, used to ask for input. It will loop until a valid integer has been entered, and will return the validated integer.

- Generic function 2 to read and validate a number between 1 and a maximum value.

  This function will have two input parameters, a string, which describes the value to be read (to be used in prompts and error messages), and the maximum value that can be entered. It will loop until a valid integer (1 to maximum) has been entered, and will return the validated integer.

- One function to read and validate the treat type.

  This function will display a menu of treat choices, along with prices of each, to the user. It will prompt for the treat type (Y, S, P, or B), and error check that the user has entered a valid choice (accepted in upper or lowercase). If not, issue an error message and re-prompt until a valid choice is entered. The function will return the validated choice in ***uppercase***.

- One function to calculate the base treat charge.

  This function will have two input parameters, the treat type and amount purchased. It will calculate and return the charge for the specific amount of treat purchased, including take out discounts.

- One function to calculate the toppings charge.

  This function will have two input parameters, the treat type and amount purchased. It will decide whether toppings are allowed (Toppings are allowed for yogurt or soft serve ice cream under take out limits, and premium ice cream).

  If toppings are allowed, this function will call generic function number 1 to read and validate the number of toppings. It will then calculate the toppings charge, and return the toppings charge, or 0 if there was no charge.

- One function to calculate the total cost for one treat (includes base charge and topping charge, if necessary).

  This function will have one input parameter, the treat type. If necessary, it will call generic function 2 to determine the amount (ounces or scoops) purchased. (Max of 64 ounces for yogurt and soft serve, and 5 scoops for premium). It will then call the above two charge functions. The function will return the total cost for one treat.

Remember that the use of global variables is **NOT** allowed. The functions must use **parameters** and **return** values to pass required data to and from each function.

- All **input** only parameters should be passed ***by value***.
- Output (***reference***) parameters are used when ***more than one*** value is being passed back. If only ***one*** value is passed back, the **return** statement should be used instead.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Program Execution

1. Display a description to the user, explaining what the program will do.

2. Implement an ***outer*** loop in **main**, to process customer orders, as follows:

   a. Call generic function number 1 to read the number treats purchased by one customer.

   b. If the function returns 0, exit the ***outer*** loop. Otherwise:

      Using the value entered by the customer, implement an ***inner*** loop in **main**, to calculate the cost for ***each treat***, as follows:

      i. Call the function to read and validate the treat choice.

      ii. Call the function to calculate the total cost for one treat.

      iii. Decide whether treat should be half cost, and adjust it accordingly.

      iv. Display the cost of the treat and add it to the total bill for the customer.

   c. After the cost of all treats for one order have been displayed (i.e. the ***inner*** loop exits), display the total bill amount.

3. The ***outer*** loop should continue to loop, until 0 is entered for the number of treats purchased by one customer.

Sample Input and Output *(continued on next page)*

```
This program will calculate customer bills for frozen treats

NEW CUSTOMER
Number of treats purchased (0 to exit): 4

Treat Choices:
   Y - Frozen Yogurt           $ 0.44 per ounce
   S - Soft Serve Ice Cream    $ 0.49 per ounce
   P - Premium Ice Cream       $ 1.62 per scoop
   B - Ice Cream Bar           $ 3.49 per bar

For treat #1, enter letter for treat type: y

Enter number of ounces: 10
Enter number of toppings: 3

Charge for treat #1 is $ 5.66

Press any key to continue . . .
```

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

```
Treat Choices:
   Y - Frozen Yogurt           $ 0.44 per ounce
   S - Soft Serve Ice Cream    $ 0.49 per ounce
   P - Premium Ice Cream       $ 1.62 per scoop
   B - Ice Cream Bar           $ 3.49 per bar

For treat #2, enter letter for treat type: s

Enter number of ounces: 30

Charge for treat #2 is $ 13.50

Press any key to continue . . .

Treat Choices:
   Y - Frozen Yogurt           $ 0.44 per ounce
   S - Soft Serve Ice Cream    $ 0.49 per ounce
   P - Premium Ice Cream       $ 1.62 per scoop
   B - Ice Cream Bar           $ 3.49 per bar

For treat #3, enter letter for treat type: B

Charge for treat #3 is $ 3.49

Press any key to continue . . .

Treat Choices:
   Y - Frozen Yogurt           $ 0.44 per ounce
   S - Soft Serve Ice Cream    $ 0.49 per ounce
   P - Premium Ice Cream       $ 1.62 per scoop
   B - Ice Cream Bar           $ 3.49 per bar

For treat #4, enter letter for treat type: p

Enter number of scoops: 3
Enter number of toppings: 1

Charge for treat #4 is $ 2.43

Bill total for 4 treats is $ 25.08

Press any key to continue . . .

NEW CUSTOMER
Number of treats purchased (0 to exit): 0
```

Testing

As part of your submission for this week, you must write a **test plan** for this program (program 1). Remember to test the border cases that generally cause errors.

Your test plan must include:

- the ***rationale*** for testing each condition within the program
- a list of the specific test cases

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Each test case must include:

- the exact ***user input used*** for the test
- the exact ***output expected***

Program #2

Mathematically, given a number $n = n_0$, we can define a sequence n_1, n_2, ... where n_{i+1} is the sum of the squares of the digits of n_i. *n* is ***happy*** if and only if there exists *i* such that $n_i = 1$.

Here is an example:

First, take each digit of any positive integer, square them and add them together.
We will use the number ***nineteen*** as an example:

19 → $1^2 + 9^2 = 82$

Then take each digit of this number, square them and add them together:

82 → $8^2 + 2^2 = 68$

And repeat until the result is 1:

68 → $6^2 + 8^2 = 100$

100 → $1^2 + 0^2 + 0^2 = 1$

So when the sum of the squares of the digits in the last member in the sequence starting with *n* eventually reduces to 1, *n* is ***happy.***

Additionally:

If a number is ***happy***, then all members of its sequence are ***happy***.
If a number is ***unhappy***, all members of its sequence are ***unhappy***.

The above example sequence: 19, 82, 68, 100
is happy, and so all of the terms within the sequence are also happy.

Write a program that determines which numbers produce a happy sequence, within a series of test numbers.

The user will provide a first number to test and a last number to test. The program will test the numbers starting with the first number, incrementing by one, and repeating until the last number.

Program Implementation Requirements:

1. Tell the user what the program does before prompting for input.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

2. **main** will call a ***user-defined*** function to read the first and last numbers for the range to test from the user.
    - The ***user-defined*** function will:
        - Separately read and validate each number. Both the first and last numbers must be ***at least*** 1 and can be ***up to*** 9999 (remember to define constants for these values!)
            - If the first number is not in the correct range, issue an error message and then re-prompt until the number entered is valid.
        - The last number cannot be smaller than the first number.
            - If the last number is not in the correct range or is smaller than the first number, issue an error message and then re-prompt until the number entered is valid.
        - Pass back both values to **main** using ***reference*** parameters.
3. **main** will start with the first number as the test number, and loop until all numbers including the last number have been tested. Each time the loop runs, **main** will:
        - Call a user-defined function to test if the number is happy.
        - Display the results (whether the sequence is happy or unhappy), and pause so the results can be read by the user.
        - Increment the test number.
    - The ***user-defined*** function to test whether a number is happy must implement ***nested loops***, as follows:
        - A test number will be passed into the function.
        - The function's ***outer loop*** will test the number, as follows:
        - Use an ***inner loop*** to compute the sum of the squares of the digits in the current term. NOTE: You must use a loop, so this program will still work for larger numbers, if the maximum value of the last number constant is modified.
        - Count and display each term.
        - Each term displayed should be formatted to a width of 8, and 10 terms should be displayed per line.
        - Repeat with the sum of the squares of the digits in the current term becoming the next term to test, until it is determined:
            - The test number is happy (when the sum of the square of the digits is 1)
            - The test number is unhappy (when 50 terms have been computed, without reaching 1).
        - After the outer loop exits, the function shall display the number of terms in the sequence.
        - A **boolean** value will be returned to **main**, indicating whether the displayed sequence is happy or not.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Sample Input/Output:

```
Program to test for Happy numbers

Enter the first number to test: 79

Enter the last number to test: 1
Invalid -- must be between 79 and 9999.  Try again.

Enter the last number to test: 82


Test sequence for 79 is:
      79     130      10       1
4 sequence terms computed

The above sequence of numbers is happy

Press any key to continue . . .


Test sequence for 80 is:
      80      64      52      29      85      89     145      42      20       4
      16      37      58      89     145      42      20       4      16      37
      58      89     145      42      20       4      16      37      58      89
     145      42      20       4      16      37      58      89     145      42
      20       4      16      37      58      89     145      42      20       4
50 sequence terms computed

The above sequence of numbers is unhappy

Press any key to continue . . .


Test sequence for 81 is:
      81      65      61      37      58      89     145      42      20       4
      16      37      58      89     145      42      20       4      16      37
      58      89     145      42      20       4      16      37      58      89
     145      42      20       4      16      37      58      89     145      42
      20       4      16      37      58      89     145      42      20       4
50 sequence terms computed

The above sequence of numbers is unhappy

Press any key to continue . . .


Test sequence for 82 is:
      82      68     100       1
4 sequence terms computed

The above sequence of numbers is happy

Press any key to continue . . .

Testing of numbers 79 to 82 complete.
```

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

NOTES:

- If the first and last numbers are the same, and only the first number will be tested.
- For testing purposes, it may help to know that the happy numbers up to 100 are:

  1, 7, 10, 13, 19, 23, 28, 31, 32, 44, 49, 68, 70, 79, 82, 86, 91, 94, 97, 100

Program and Function Documentation (both programs)

1) Include **top of program** comments as specified in course Content section 1.8.

2) Comment any constants/variables whose names are not completely descriptive.

3) Include prototypes for ***all*** of your user-defined functions. The prototypes should be placed above **main**, and the function definitions should be placed below **main**.

4) Provide a comment above **each user-defined function definition** that includes:

- Dividing lines
- A description of what the function does
- The name and a description of each ***input parameter***, used to pass values into the function
- The name and a description of each ***output parameter***, used to pass values back from the function
- The name (if there is one), and a description of what is returned from the function via the **return** statement (if anything).

Examples:

```
// ---------------------------------------------------------
// Task:  Reads a positive and negative number from the user
//        and error checks them
// Input parameters: prompt - prompt to display
// Output parameters: posValue, negValue - valid numbers read
// ---------------------------------------------------------
void getData (string prompt, int& posValue, int& negValue)
```

Submission

This programming assignment is due by midnight Sunday. Submit your program source code to the ***Prog Assn 6*** folder in the ***Dropbox*** area of the online course. **All** files should be attached to **one** submission for programming assignment 6.

Before submitting your program files, you MUST name them as follows:

Lastname-wk6-prog1.cpp
Lastname-wk6-prog2.cpp

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

And your test plan file name MUST be:

Lastname-wk6-p1testplan.xxx

For example:

Smith-wk6-prog1.cpp
Smith-wk6-prog2.cpp
Smith-wk6-p1testplan.doc

Grading

The rubric that will be used to grade your program is included on the next pages.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

CS 361 Control Structures
Programming Assignment 6 Rubric

Program 1: (52 pts)				
Rating **Rating Category**	**Exemplary**	**Proficient**	**Basic (needs work)**	**Not Demonstrated**
Data Storage Points	Constants defined for ALL fixed values. Variables defined locally, using appropriate data types. Followed all coding standards naming conventions.	A few minor errors in data typing, naming, or assignment OR missing one or two necessary constants for fixed value.	One or more major errors, OR majority of names did not follow naming conventions, OR missing multiple constants for fixed values.	Constants not used, identifier names are not descriptive, and there is a multitude of errors OR defined global variable(s).
Function Implementation: User Input from Menu	Function defined and called as specified by the requirements. Displays menu using constants for fixed values. Validates all user input. Passes back results.	One or two minor errors in function definition or call, or in the user input/validation or error messages, or menu display uses hardcoded values instead of constant names.	Many errors in function definition or call, or in the user input and validation code or error messages.	Required function was not implemented. Task was simply performed from within the main function.
Function Implementation: Other User Input	Generic function(s) defined and called as specified by the requirements. Validates user input correctly, with descriptive error messages.	One or two minor errors in function definition or call, or in the user input/validation or error messages.	Many errors in function definition or call, or in the user input and validation code or error messages. Or two separate functions defined.	Required function(s) not implemented. Task was simply performed from within the main function.
Function Implementation: Calculations	Functions defined and called as specified by the requirements, with proper parameter usage and a return values. All math computations use constants for fixed values and compute correct results. All control statements written correctly.	One or two minor errors in function definitions or calls, or in the math computations, or in the control statements.	Many errors in function definitions or calls, or in the math computations, or in the control statements.	Required functions were not implemented. Task was simply performed from within the main function.
Output	Displays all of the required output for each calculation, formatted according to program requirements.	One or two minor errors in the output display.	Many errors in the output display.	No output.
main function: Program Logic and looping	The program logic accomplishes all program requirements and uses ***nested*** loops correctly.	One or two minor errors in the main function logic or calls to other functions.	Many errors in the main function logic, OR program runs correctly ONCE, but does not loop.	Logic accomplishes few or none of the program requirements.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

C++ Constructs / Readability / Miscellaneous	Code is organized and easy to follow. There is no unnecessary code and all code is spaced and indented correctly.	Code is fairly easy to read, but there are some spacing, indentation, and/or other minor issues, or extraneous code.	Code is somewhat difficult to read or has some other major issue.	The code is formatted so badly that it is extremely hard to read or has multiple major other issues.
Test Plan	Test plan includes rationale and tests that validate ALL logic.	Missing parts of rationale or missing tests of some logic.	Missing all rationale and/or missing many tests of the logic.	No test plan submitted.

Program 2: (48 pts)

Rating **Rating Category**	**Exemplary**	**Proficient**	**Basic (needs work)**	**Not Demonstrated**
Data Storage and main function logic	Constants defined for ALL fixed values. Variables defined locally, using appropriate data types. The logic accomplishes all program requirements.	One or two minor errors in data typing, naming, or assignment, OR in the main function logic or calls to other functions.	Many errors in data typing, naming, or assignment, OR in the main function logic or calls to other functions.	Logic accomplishes few or none of the program requirements.
Program Input	Displays a descriptive prompt for each input. Validates all user input using constants for valid ranges, and re-prompting until good values are entered.	Reads input, but does not validate one of the values correctly.	Reads input, but does not validate any of the values correctly.	Hardcoded values used instead of reading input, OR reading of input fails completely.
Program Logic: loops and calculations	All math computations compute correct results. All if and nested loop statements are written correctly, with no use of break, return, or exit to exit a loop.	One or two minor errors in math computations, if statement logic, and/or nested loop logic.	Many errors in math computations, if statement logic, and/or nested loop logic.	Few or none of the calculations are correct.
Output	Displays all of the required output, formatted according to program requirements.	One or two minor errors in the output display.	Multiple errors in the output display.	Displays completely incorrect output, or does not display any output at all.
Program Modularity	Program logically broken down into at least main + required functions. All functions defined and called correctly, with proper parameter passing.	One or two minor errors in the logical breakdown or in function definitions or calls, OR missing at least one required function.	Many errors in the logical breakdown or in function definitions or calls, OR missing multiple required functions.	There is no logic to the code included in each function, OR none of the required functions were implemented.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

C++ Constructs / Readability / Miscellaneous	Code is organized and easy to follow. There is no unnecessary code and all code is spaced and indented correctly.	Code is fairly easy to read, but there are some spacing, indentation, and/or other minor issues, or extraneous code.	Code is somewhat difficult to read or has some other major issue.	The code is formatted so badly that it is extremely hard to read or has multiple major other issues.

Compilation	Code compiles correctly	One minor error prevents code from compiling	Several minor errors or one major error prevent code from compiling	Many minor or major errors prevent code from compiling
Documentation	Top of program comments include all required components. Top of function comments explain what function will do, and describe all parameters and return values. Inline comments are included as necessary.	Program description missing some details about the input, output, or processing OR top of function comments vague or missing some details about parameters and return values OR missing other minor necessary comment.	Documentation has many problems with missing, incomplete and/or incorrect items in program description, top of function comments, and/or inline comments.	Only a few (or no) comments in program.

NOTE: Deductions for late submissions will be applied (-2% per day late)
But programs will **not be accepted** if they are submitted **more than 5 days** late (grade will be 0).

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law